TeleHSupport: A Multimodal, Literacy-Aware AI-enabled Kiosk for User-Centered Care Planning

Faryaneh Poursardar*, Saumya Dabhi*, Ariana Montemayor†, Connor Haislip†,
Janelle Clarke†, Grace Barthold‡, Tina Gustin†‡
*Health, Education, AI & Learning (HEAL), Office of Enterprise Research & Innovation, Old Dominion University, Suffolk, VA, USA
†Ellmer School of Nursing, Macon & Joan Brock Virginia Health Sciences, Old Dominion University, Virginia Beach, VA, USA
‡ODU Community Care Clinic, Center for Telehealth, Innovation, Education, and Research (C-TIER), Virginia Beach, VA, USA
Emails: {fpoursar, s1dabhi, amont008, chais001, jclar109, egustin, tgustin}@odu.edu

***Abstract*—This paper presents the prototype of an off-hours, multimodal, AI-enabled kiosk for a free clinic, embedded within a community non-profit, serving people experiencing homelessness (PEH). It uses a two-stage architecture with a Streamlit frontend for user interaction and a FastAPI backend for symptom processing and plan generation. The interface supports both speech-based and selection-based interaction for symptom collection. It follows a rule-based conversational flow to gather symptoms. The collected information is summarized and passed to an AI-based planning component, which generates an actionable plan. The interface incorporates accessibility-oriented design features for better patient interaction.**

***Index Terms*—AI-powered kiosk, multimodal, user-centric design, care planning, unhoused population, LLM**

I. INTRODUCTION

Community-based clinics provide essential medical services to unhoused populations, serving as the primary point of contact for their health needs and long-term support. For many patients, these clinics represent a rare source of trust in a fragmented healthcare system. However, these resources are usually available only during working hours. When a clinic is closed, patients may still experience symptoms and may not know what to do next. In those circumstances, the emergency room becomes the only clear option, even when the symptom may not require emergency care. This can create unnecessary ER visits and increase pressure on emergency services. To address this, we designed a kiosk prototype that can be placed outside the clinic and used during off-hours. The goal of the kiosk is to help patients describe their symptoms and receive an actionable plan in simple language. Our aim is not to replace a clinician or provide a medical diagnosis, but to offer a practical support tool that helps patients make better care decisions when the clinic is closed. This kiosk is a patient-facing system that supports both speech and selection-based interaction. It follows a rule-based flow that collects symptoms one at a time and asks focused follow-up questions before generating a recommendation on next steps to follow. A Large Language Model (LLM) is used to generate this recommendation. Many patients may have different levels of health literacy and reading or writing ability. Some of them are not comfortable with digital tools. For that reason, the interface was designed for accessibility using large text, a simple visual layout, limited colors, and speech-to-text (STT) and text-to-speech (TTS) features. This work presents the motivation and design of the kiosk as a practical intelligent system for extending clinic support beyond operating hours and improving off-hours care guidance for PEH.

II. RELATED WORK

Symptom-checker tools use structured, sequential interfaces for symptom intake. Publicly available tools such as Mayo Clinic, Ada, and Buoy guide users through a series of symptom-related questions in order to narrow possible causes or determine the appropriate level of care [1, 2, 3]. Although some systems present this workflow in a questionnaire format and others in a chatbot-like style, the underlying interaction remains largely the same, where the user provides an initial symptom and then answers follow-up questions presented by the system. This pattern is also reflected in the literature that compares the diagnostic performance of these tools, where symptom checkers are described as patient-facing tools in which users begin by entering a chief complaint and then answer follow-up questions posed by the system [4]. Building on this design tradition, our system also performs symptom intake one question at a time, but introduces speech-supported interaction into the process. Specifically, the system can read questions aloud using TTS and accept spoken responses through STT, while still allowing users to respond through selectable options. In this way, our interface retains the structured symptom-gathering approach used in earlier symptom checkers and broadens the modes of interaction beyond conventional text-heavy input.

III. ARCHITECTURE

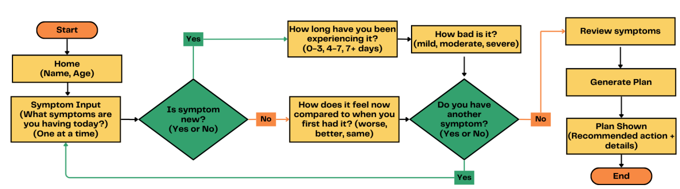

Fig. 1. Rule-based interaction flow of the kiosk prototype

A. System Design and Interaction Flow

The frontend is implemented with Streamlit, and the backend with the FastAPI framework. The methodology can be viewed as a two-stage approach. In the first stage, it collects structured symptom information from the user through a guided conversation. As illustrated in the flowchart figure 1, the interaction starts with the collection of basic user information, followed by symptom entry one symptom at a time. For each symptom, we used a rule-based question flow to gather relevant details, including whether the symptom is new, its duration, its severity, or whether it has changed over time. This process is repeated for additional symptoms if needed, after which the user reviews the collected information. In the second stage, the structured symptom summary is passed to the AI-based planning component, which transforms it into an easy-to-read action plan. The resulting output is then shown to the user as a simple recommendation for the next appropriate step.

B. User-Centered Interface Design

An important part of the kiosk is the user-centered interface design. All panels are shown in Appendix B. The panels use short prompts, large buttons, limited response options, and repeated layout patterns. The same basic interaction style is used throughout the whole conversation. This style reduces the cognitive burden on the user by showing them one panel at a time. Each question appears on its own screen, and the user moves through the interaction using simple Back, Next, Confirm, and End buttons. This consistent design helps users learn the system as they go. Once they understand one panel, the next panels feel familiar.

We conducted an Institutional Review Board (IRB) approved survey to gather patient input on what features they would like in the kiosk. The survey showed that some patients wanted access to their Electronic Health Records (EHR) through the local MyChart system. However, many do not use personal cell phones, and some who have phones cannot afford internet connectivity to access their health information. To address this need, we added buttons that link to MyChart and a MyChart user guide. Figure 2 shows the welcome panel with those buttons. Because the kiosk will have internet access, it can provide direct access to the EHR system. Clicking the “Open MyChart” button directs the user to the EHR website, where they can log in and view health records such as lab results and upcoming appointments. The “MyChart User Guide” button opens a PDF document that explains how to use the MyChart system. In this way, the kiosk also supports patients in accessing their health records.

C. Input Modalities: Type and Speech

A central design choice in the prototype is the use of two input modes: option selection and speech input. Nearly every key panel in the workflow provides both options. For example, in figure 3, when the system asks what symptom the user is having, they can either choose a symptom from a list or speak the symptom using the voice interface (figure 4). The same pattern is used later for yes or no questions, duration questions, severity questions, and comparison questions. This dual-input design allows users to choose the method that feels easier to them. It also supports users who may have difficulty selecting, users who prefer speaking, or users in settings where one mode may be easier than the other.

The speech-based interaction is designed with confirmation at each step. After the user speaks, the system shows what it heard and how it interprets the answer. For instance, as depicted in figure 5, if a user said “fever,” the system displays the transcript and then shows the matched symptom as “Detected: Fever.” If the speech input could not be matched correctly, the system tells the user that it could not match the response and suggest trying again or using the select input mode instead. This confirmation process reduces the chance that the system would continue based on an incorrect interpretation. It shows the recognized result to the user so that they can verify it before moving forward. This STT functionality is implemented using Python’s PyAudio and SpeechRecognition libraries. PyAudio is used to capture audio of a user from local microphone. It saves this audio in WAV format. Then the SpeechRecognition library is used to transcribe the recorded WAV into text using the Google Web Speech API. This text is then used to map to the actual options for a question.

D. Text-to-Speech (TTS) Support

A TTS function is included to read questions aloud to the user. This serves as an accessibility feature for users who may have difficulty reading on-screen text. Whenever a new panel appears, the question displayed in that panel is spoken aloud. To implement this feature, we used the offline TTS engine pyttsx3, which relies on OS-native voices. During development on Windows 11, the Zira voice was used through the registry token. Using an offline TTS engine helps avoid the latency that may arise when external TTS services are accessed through API calls.

E. Symptom Collection Strategy

The symptom collection strategy and follow-up questions with options were designed with guidance from our nurse clinician partners who practice at the community clinic and are experienced in caring for unhoused patient population. Therefore, this rule-based flow reflects the practical needs of the clinic. Many users may present with more than one symptom, but asking about all symptoms at once can becomes confusing. Hence, the symptom collection process is organized around one symptom at a time. The flowchart 1 shows that after entering one symptom, the user is asked whether the symptom is new. If the symptom is new, the system collects both duration and severity. If the symptom is ongoing or chronic, the system instead asks how the symptom feels now compared to when it first started. For a new symptom, duration is captured using a limited set of categories: 0-3 days, 4-7 days, and 7+ days (figure 7). Symptom severity is captured through the question, “How bad is the symptom?” (figure 9) with three response options: mild, moderate, and severe.

For existing or chronic symptoms, the system uses a simpler comparison question: whether the symptom felt worse, better, or the same compared to when it first occurred (figure 10). This is a practical design because users may not always know exact timelines or severity changes for chronic conditions, but they can describe whether things are improving or getting worse. This will be especially useful in the future when we connect the kiosk with the patient's health record to understand the severity of previous occurrences. The prototype also supports cases where the user has more than one symptom. After processing one symptom, the system asks if there is another symptom (figure 11). If the user selects yes, the flow returns to the symptom input panel, and the process repeats. This loop allows the kiosk to build a symptom list that includes multiple entries, each with its own related details. The review screen then presents these symptoms back to the user in summary form before plan generation (figure 12).

F. Plan Generation Process

Once the user confirms the symptom summary, the system enters a loading stage by showing a spinner, along with a sentence telling them to wait while we work on their data to generate plan (figure 13). While this is showing on the interface, the request carrying the symptom payload is sent to the backend for further processing. The backend is built using Python's FastAPI service. We used OpenAI's GPT-5 [5] model to generate a plan. Using a structured prompt, as shown in Appendix A, supplied along with the symptoms, we make an API call to GPT-5. The response is a report that is organized into short and practical sections containing a recommended action, a brief explanation, what the user should do next, and warning signs to watch for.

G. Recommendation Categories and Output Design

The final output of the system is limited to three decision categories: self-treatment, wait to be seen, and go to the emergency room. These categories are shown with different colored recommendation boxes, and each category was paired with short supportive text (figure 14). The self-treatment category suggests that the condition might be manageable for now, but the system still advises follow-up at the clinic so the patient can be checked for any potentially serious condition. The wait to be seen category suggests that urgent emergency care might not be needed, but that the patient should seek clinic care soon or return when the clinic opens. The go to the ER category is used for situations that could be serious and require immediate action. Limiting the output to three categories was an important design and methodology decision. Too many categories or too much medical detail could create confusion. The additional supporting sections, such as "what you should do next" and "warning signs" makes the recommendation more useful without making it complicated (figure 15). That shows what the patient should do next, depending upon the category. We used a robust prompt for the GPT-5 to avoid using complex medical jargon by making the language simple and understandable to patients, as well as to avoid prescribing medication. Moreover, we added safety, language, and output rules to deal with failure cases and hallucination risk.

IV. Limitations and Future Work

The kiosk prototype shows a useful approach for off-hours symptom guidance, but it has some limitations. First, while the current design is based on initial user interviews with 30 participants whose feedback was incorporated into the system as well as guided by our nursing collaborators' experience with unhoused populations, future work will involve in-situ evaluation with patients to assess usability, trust, and real-world impact. Second, the final recommendation is generated by an LLM-based component. The prompt includes safety, language, and output rules, but concerns may remain about consistency in the suggested next step. Third, the kiosk collects only a limited amount of symptom information through a short rule-based flow. Although this keeps the interaction simple, it may not provide enough detail for a deeper conclusion. Future work will explore how to gather more information without making the system harder to use.

V. Conclusion

This paper presented the design of an off-hours, AI-powered kiosk prototype for a community clinic serving PEH. The kiosk was developed to provide simple and actionable symptom-guided care recommendations when the clinic is closed. The system was designed to support accessibility and reduce user burden by combining a rule-based symptom collection flow with multimodal interaction. Overall, this work shows how a patient-facing intelligent system can help extend clinic support beyond operating hours, particularly for people experiencing homelessness.

Acknowledgment

This work was supported in part by the National Science Foundation (NSF) under Grant No. 2426781.

References

[1] Mayo Clinic. *Symptom Checker*. https://www.mayoclinic.org/symptom-checker/select-symptom/itt-20009075. (accessed: January 16, 2026).

[2] Ada Health. *Check your symptoms now. Built with doctors.* https://ada.com/symptom-assessment-lite/. (accessed: January 16, 2026).

[3] Buoy Health. *Use our symptom checker to learn why you're not feeling well.* https://www.buoyhealth.com/symptom-checker/. (accessed: January 16, 2026).

[4] Mohammad Hammoud et al. "Evaluating the diagnostic performance of symptom checkers: clinical vignette study". In: *Jmir ai* 3.1 (2024), e46875.

[5] OpenAI. *Introducing GPT-5*. https://openai.com/index/introducing-gpt-5/. (accessed: February 10, 2026).

Appendix

A. Prompt Template

```
You are a medical triage helper. You are NOT a
    doctor and you do NOT diagnose.
You must produce a structured plan using ONLY the
    information given by the user.

Your tasks:
1) Help decide how urgent the medical situation is
    based on the symptoms described, and give safe
    advice on what to do next.
```

```
   You should be very cautious and prioritize safety
       , especially since you have limited
       information.
2) Choose ONE action level:
   self_treatment | wait_to_be_seen | go_to_er

Action level definitions (choose ONE):
- self_treatment:
  Mild symptoms. No danger signs. Safe simple steps
      are enough for now.

- wait_to_be_seen:
  Not urgent right now. Symptoms should be watched.
  Tell what to watch for and when to get help.

- go_to_er:
  Any danger signs OR you cannot rule out a serious
      problem.

Safety rule:
- If symptoms could be a serious heart, lung, brain,
    or bleeding problem, choose go_to_er even if
    the condition list does not include those
    diseases.
- Do NOT mention any medicines, pills, drugs,
    dosages, or brand names.
- Do NOT say  take  aspirin,  take
    ibuprofen,  use  antibiotics, etc.
- Only suggest non-medicine actions like: rest,
    drink water, go to clinic/ER, call 911, ask
    staff for help.
- The patient you are helping are homeless who do
    not have access to many basic things so write
    your response accordingly by avoiding mentioning
     of house, place etc.

Language rules (VERY IMPORTANT):
- Write like you are talking to a 5th grader.
- Use short sentences.
- Avoid medical words.
- Assume the person may not have money, a phone,
    transportation, or a safe place to rest.
- Give practical steps that work in that situation.

Output rules:
- Return ONLY a JSON object with these keys exactly:
  - triage_level (one of: self_treatment,
      wait_to_be_seen, go_to_er)
  - level_explanation (1-2 short sentences)
  - follow_up_actions (1-2 short bullets)
  - watch_outs (1-2 short bullets)
- If information is missing, make safe, cautious
    suggestions and include watch-outs.
```

B. Interface

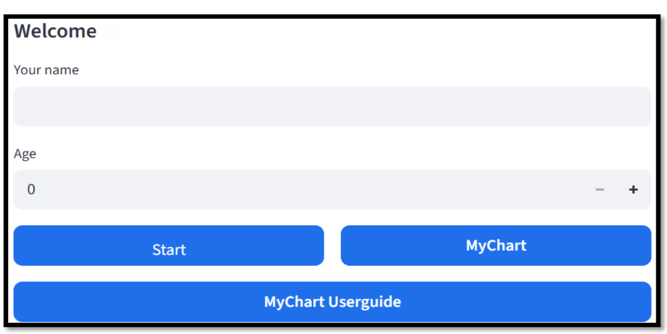

Fig. 2. Welcome Panel

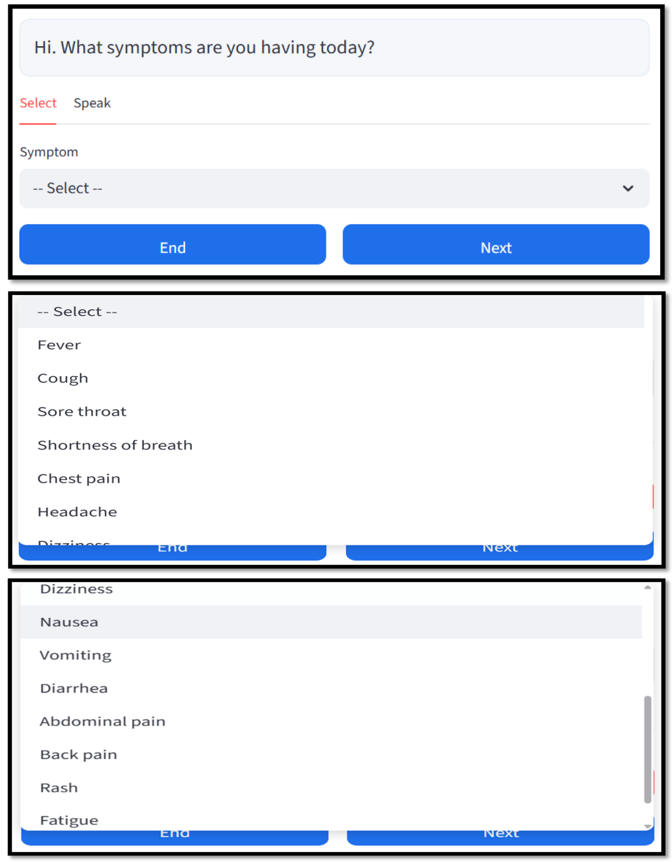

Fig. 3. Symptoms entry panel with a list of symptoms in the select tab

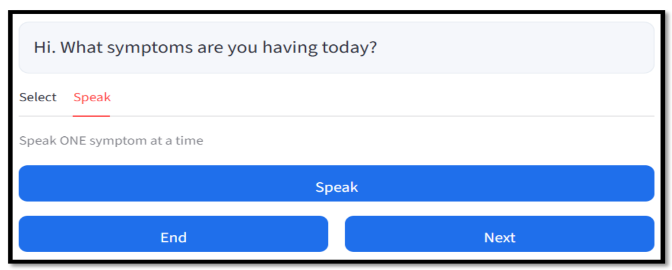

Fig. 4. Symptoms entry panel with the speak tab open

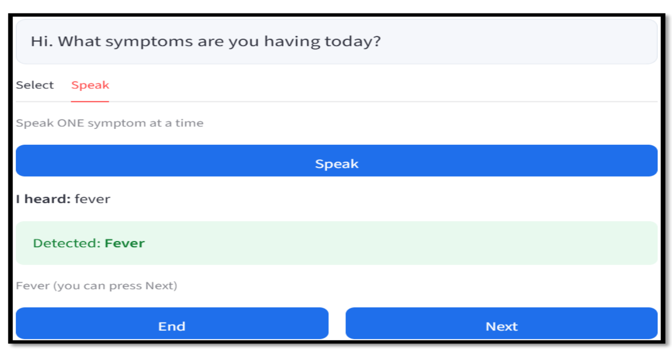

Fig. 5. Panel showing spoken symptoms

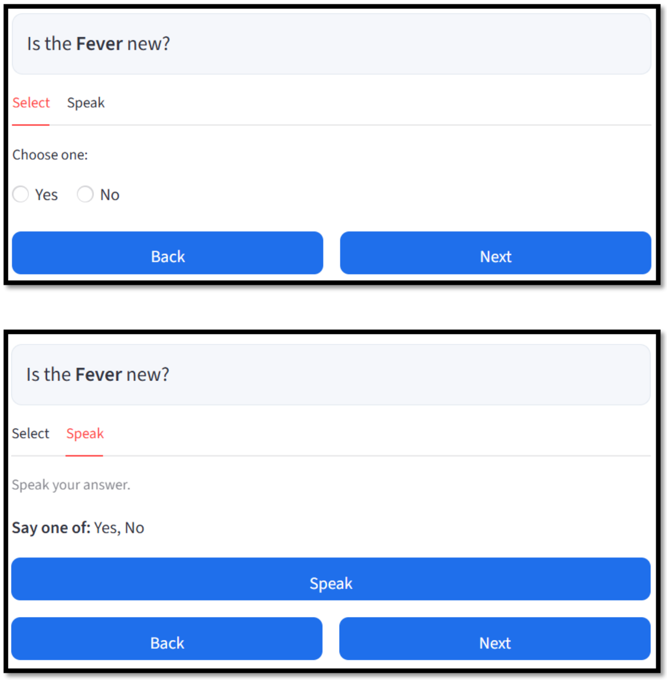

Fig. 6. Panels inquiring whether symptoms are new

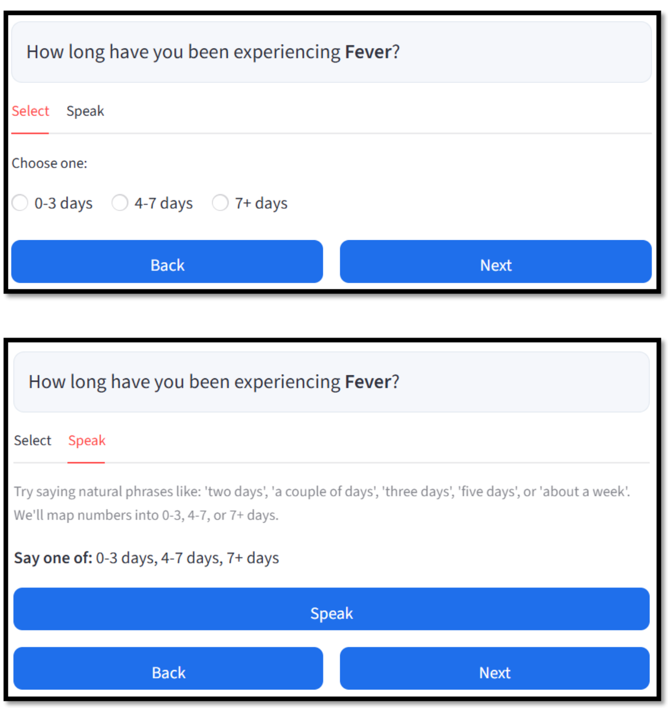

Fig. 7. Panels asking for the duration of new symptom

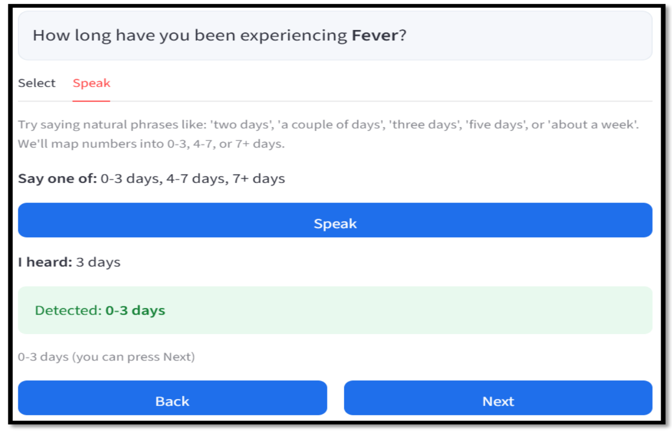

Fig. 8. Panel showing mapped option to duration bucket

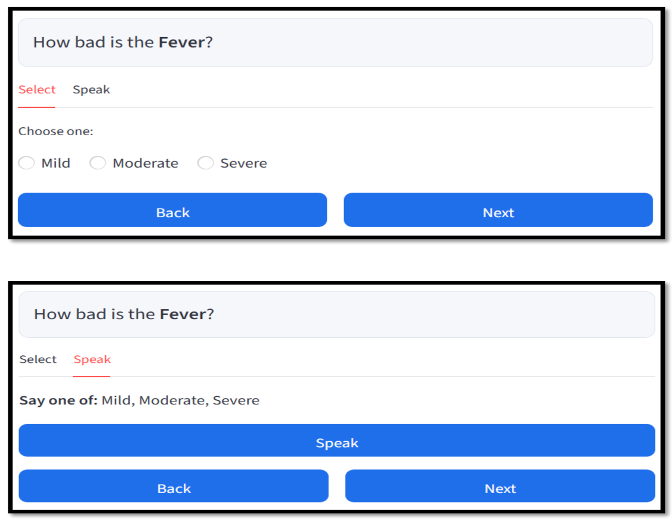

Fig. 9. Panels assessing severity of new symptom

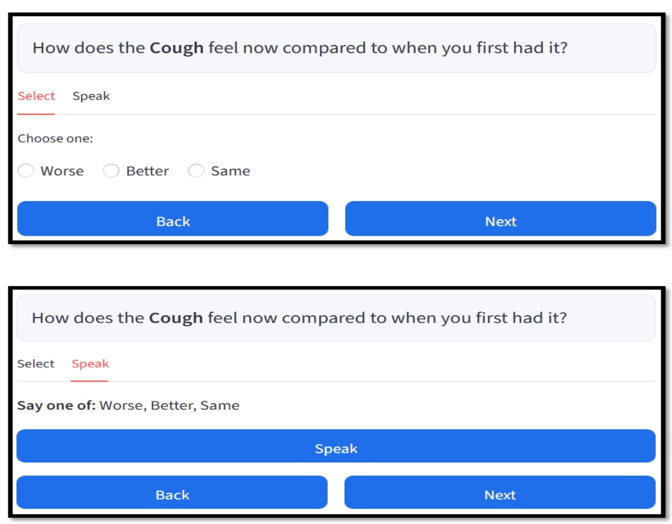

Fig. 10. Panels assessing the severity of chronic symptom

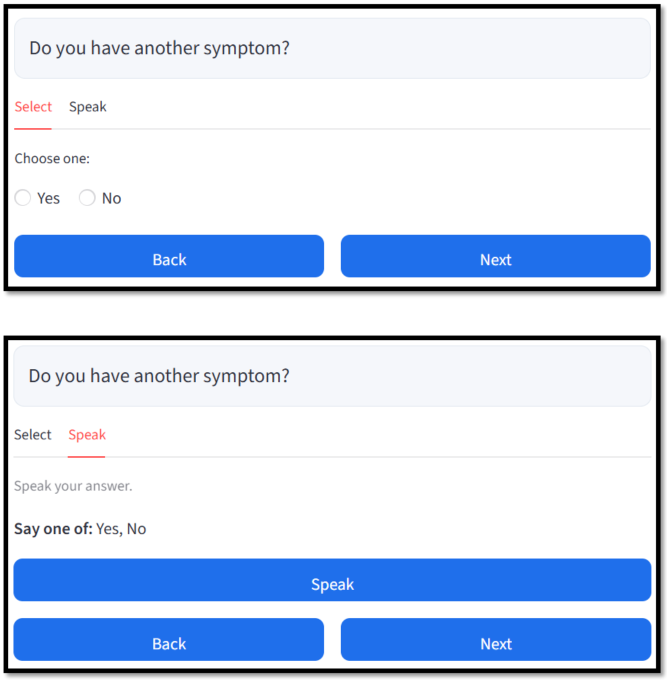

Fig. 11. Panels checking for more symptoms

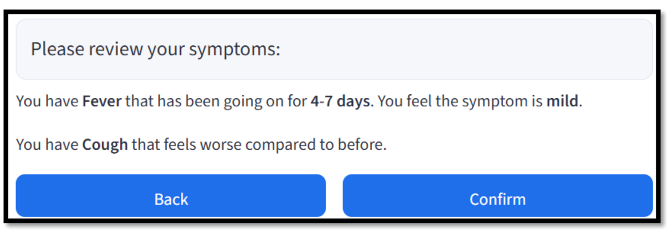

Fig. 12. Review Panel

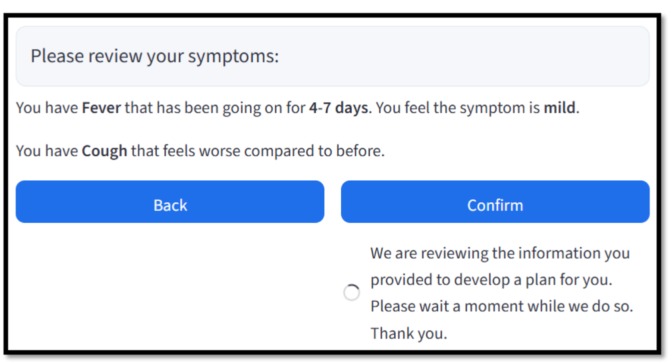

Fig. 13. Panel showing spinner when pressed confirm

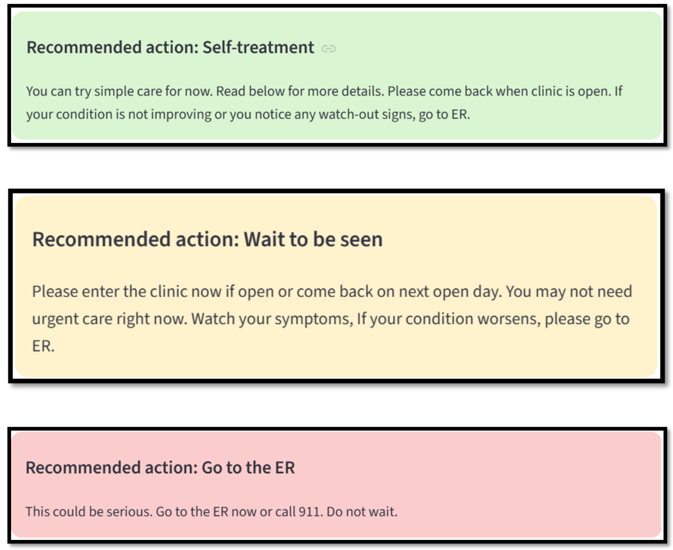

Fig. 14. Final recommendation output in 3 categories

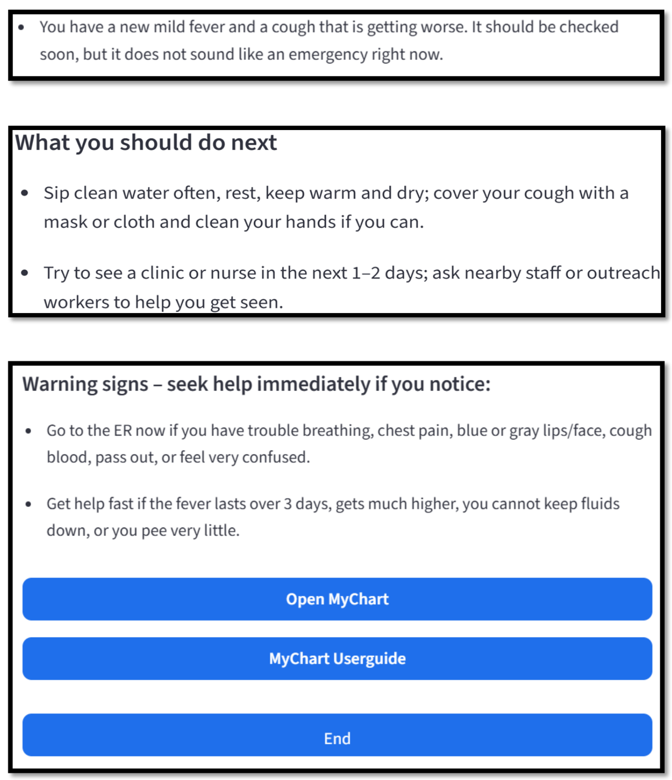

Fig. 15. Follow-up steps